

SMART BLACK BOX LOGGER AND EVENT RECORDER USING LPC 1768

ABSTRACT

The current project aims to develop a Smart Black Box Monitoring and Event Logging System for Electric Vehicles to enhance vehicle safety, real-time monitoring, and event recording. This system integrates the LPC1768 microcontroller, keypad interface, Real Time Clock (RTC) using I²C communication, LCD display, LED indicators, buzzer alert system, and UART communication to monitor critical vehicle parameters and record abnormal events. The system continuously checks parameters such as temperature, voltage, current, vibration, RPM, and battery level, ensuring reliable monitoring of vehicle conditions.

One of the major challenges in modern electric vehicles is the lack of real-time monitoring and event recording for abnormal operating conditions. Sudden faults such as overheating, high current draw, excessive vibration, or low battery levels can lead to vehicle damage or unsafe driving conditions. To address this issue, the proposed system continuously monitors vehicle parameters and compares them with predefined safety thresholds. Whenever a parameter exceeds its safe limit, the system instantly activates LED indicators and a buzzer alarm, alerting the driver about the abnormal condition.

The LPC1768 microcontroller, based on the ARM Cortex-M3 architecture, plays a crucial role in processing parameter inputs, performing threshold comparisons, and controlling system outputs. It manages data acquisition, system alerts, and communication processes efficiently, ensuring quick response and reliable operation.

The system also incorporates a Real Time Clock (RTC) module connected through the I²C communication protocol to record the exact date and time of each event, enabling accurate event logging similar to a vehicle black box recorder. All monitored data and alert events are transmitted through UART communication, allowing the system to log and store important information for later analysis.

The LCD display acts as a simple user interface, showing the selected parameter values and system status in real time. This allows drivers or technicians to easily monitor the vehicle's operating conditions. By providing continuous monitoring, timely alerts, event logging, and reliable communication through I²C and UART interfaces, the proposed Smart Black Box system improves vehicle safety, diagnostics, and operational reliability, making it a useful solution for modern electric vehicle monitoring applications.

CONTENTS		
CHAPTER NO	CHAPTER NAME	PAGE NO
CHAPTER-1	INTRODUCTION	1
	1.1 Introduction	1
	1.2 History	2
	1.3 Advantages	3
	1.4 Problem Statement	3
CHAPTER-2	LITERATURE OF SURVEY	4
	2.1 Existing Systems	4
	2.2 Need for a New System	4
CHAPTER-3	ARM cortex LPC1768	7
	3.1 Introduction of LPC1768	7
	3.2 Pin Diagram of LPC1768	8
	3.3 Features	9
	3.4 Pin description	9
	3.5 LPC1768 specifications	12
	3.6 Why LPC1768?	13
	3.7 Advantages and Disadvantages	13
CHAPTER-4	HARDWARE IMPLEMENTATION	14
	4.1 Block Diagram and Schematic Diagram	14
	4.2 Components and circuit connections	16
	4.3 Working mechanism	23
CHAPTER-5	SOFTWARE IMPLEMENTATION	25
	5.1 Keil uVision IDE	25
	5.2 Embedded C Programming	26
	5.3 GPIO Interfacing	27
	5.4 Flash Magic	28
	5.5 Project Code	29

CHAPTER-6	RESULTS	35
	6.1 Parameter Selection Using Keypad	35
	6.2 LCD Readings	38
	6.3 UART Event Logging Output	39
CHAPTER-7	CONCLUSION AND FUTURE SCOPE	40
	7.1 Conclusion	40
	7.2 Future Enhancements	41
	REFERENCES	42
	APPENDIX OVERVIEW OF TOOL	43
	Keil uVision IDE	43
	Flash Magic	43

LIST OF FIGURES		
FIGURE NO	NAME OF THE FIGURE	PAGE NO
3.1	LPC1768 Block Diagram	8
4.1	Project Block Diagram	14
4.2	Schematic Diagram	15
4.3	Buzzer	17
4.4	LCD Display	18
4.5	Array of LED's	19
4.6	4 x 4 Keypad Module	20
4.7	Real Time Clock Module	21
4.8	UART Communication Interface	22
5.1	Selecting LPC1768 as the Target	25
5.2	Writing and building the code to the target	26
5.3	Dumping the code using Flash Magic	28
6.1	System Initialization Display	35
6.2	Temperature Monitoring Display	36
6.3	Pressure Normal Condition	36
6.4	Pressure Alert Condition	37
6.5	Humidity Monitoring Display	38
6.6	Voltage Normal Condition	38
6.7	Voltage Alert Condition	39
6.8	Current Monitoring Display	39
6.9	Vibration Monitoring Display	40
6.10	RPM Monitoring Display	40
6.11	Fuel Level Monitoring Display	41
6.12	UART Event Logging Output	42

LIST OF TABLES		
TABLE NO	NAME OF THE TABLE	PAGE NO
3.1	Port and Pin Selection Table	12
3.2	Specifications of LPC1768 Microcontroller	12
4.1	Specifications of Buzzer	16
4.2	Pin description of IR Buzzer	16
4.3	Specifications of LCD Display	17
4.4	Pin description of LCD Display	17
4.5	Specifications of LED Indicators	18
4.6	Pin description of LED Indicators	18
4.7	Specifications of 4x4 Keypad Module	19
4.8	Pin description of 4x4 Keypad Module	19
4.9	Specifications of RTC Module	20
4.10	Pin description of RTC Module	20
4.11	Specifications of UART Communication	21
4.12	Pin description of UART Communication	21
4.15	Circuit connections	23

CHAPTER 1

INTRODUCTION

1.1 Introduction

Modern vehicles, especially electric vehicles, rely heavily on electronic systems to ensure safe, efficient, and reliable operation. Monitoring critical vehicle parameters in real time is essential to prevent system failures, detect abnormal conditions, and improve overall vehicle safety. A Smart Black Box Monitoring and Event Logging System plays an important role in recording important vehicle data and providing alerts when unsafe conditions occur.

At the heart of this system is the LPC1768 microcontroller, an ARM Cortex-M3 based processor capable of handling multiple tasks efficiently. The LPC1768 is responsible for monitoring different system parameters, processing input data, performing threshold analysis, and controlling output devices such as LEDs, buzzers, and display units. With its high processing capability and reliable peripheral support, the microcontroller ensures smooth operation of the monitoring and logging system.

The proposed system continuously monitors several important vehicle parameters such as temperature, voltage, current, vibration, motor speed (RPM), and battery level. These parameters represent the health and operational status of an electric vehicle. By continuously analyzing these parameters, the system can detect abnormal conditions such as overheating, excessive current draw, high vibration levels, or low battery charge. Whenever a parameter exceeds its predefined safe threshold, the system immediately activates an alert mechanism.

To ensure accurate event recording, the system integrates a Real Time Clock (RTC) module that communicates with the microcontroller through the I²C protocol, providing precise date and time information. Whenever an abnormal condition occurs, the system logs the event along with the exact timestamp. This feature allows the system to function similarly to a vehicle black box, which records important events for diagnostics, maintenance, and safety analysis.

A keypad interface is incorporated to allow the user to select different system parameters for monitoring. The selected parameter values are displayed on a 16×2 LCD display, which serves as a simple and user-friendly interface. The LCD continuously shows real-time parameter values along with system status messages such as normal operation or alert conditions. This allows drivers or technicians to easily monitor the vehicle's operational status.

In addition to visual indications, the system also provides audible and visual alerts using a buzzer and LED indicators. When a parameter crosses the defined safety threshold, the corresponding LED is activated and the buzzer generates an alert sound. This immediate

warning helps users quickly identify potential faults and take corrective actions before serious damage occurs.

Furthermore, the system supports UART communication, which allows vehicle data and event logs to be transmitted to an external computer or monitoring device for analysis. The integration of I²C communication for RTC interfacing and UART communication for data logging enables reliable communication between system components and external monitoring devices.

The proposed Smart Black Box Monitoring System is designed to be cost-effective, reliable, and easy to implement using widely available electronic components. By combining real-time parameter monitoring, event logging, alert mechanisms, and communication protocols such as I²C and UART, the system enhances vehicle safety and provides valuable operational insights. Such a system can be particularly useful in modern electric vehicles where electronic monitoring and fault detection play a crucial role in maintaining vehicle performance and safety.

1.2 History

The concept of a black box system has evolved significantly over time to improve safety, monitoring, and analysis in transportation systems. The earliest use of black box technology can be traced back to the aviation industry during the mid-20th century. Aircraft black boxes were introduced to record important flight parameters and cockpit conversations, enabling investigators to analyze accidents and understand the causes of failures. These systems greatly improved aviation safety by providing reliable data after critical events.

During the 1960s and 1970s, flight data recorders and cockpit voice recorders became standard equipment in commercial aircraft. These systems recorded parameters such as altitude, speed, engine performance, and pilot communication. The data collected from these devices played a vital role in accident investigations and helped engineers design safer aircraft systems. Over time, advancements in electronics and digital storage technologies improved the reliability and data capacity of black box recorders.

With the rapid growth of the automobile industry, researchers began exploring similar monitoring systems for vehicles. In the late 1980s and early 1990s, early versions of vehicle event data recorders (EDRs) were introduced. These systems recorded basic parameters such as vehicle speed, braking activity, and engine performance. Although these early systems were limited in functionality, they provided valuable insights into vehicle behavior during accidents.

The advancement of embedded systems and microcontroller technology in the late 1990s and early 2000s enabled the development of more intelligent monitoring systems. Microcontrollers such as ARM-based processors allowed vehicles to monitor multiple parameters simultaneously and respond quickly to abnormal conditions. These systems improved vehicle diagnostics and helped manufacturers detect faults in real time.

In recent years, the rapid development of electric vehicles (EVs) has increased the need for advanced monitoring and data recording systems. Electric vehicles rely heavily on electronic components such as battery management systems, motor controllers, and power electronics. Monitoring parameters such as battery voltage, current, temperature, vibration, and motor speed has become essential to ensure safe and efficient operation.

Modern embedded controllers like the LPC1768 microcontroller, which is based on the ARM Cortex-M3 architecture, provide powerful processing capabilities for real-time monitoring applications. These controllers can efficiently handle sensor inputs, perform threshold analysis, and control output devices such as displays, alarms, and communication interfaces. Their ability to process data quickly makes them suitable for vehicle monitoring and diagnostic systems.

Another major development in modern monitoring systems is the integration of Real Time Clock (RTC) modules and communication interfaces such as I²C and UART. The I²C protocol is widely used for connecting peripheral devices like RTC modules with microcontrollers, enabling reliable communication and data exchange. RTC modules allow systems to record the exact date and time of events, which is important for event analysis and fault tracking, while UART enables the transmission of logged data to external computers or monitoring systems for further analysis.

In addition, modern vehicle monitoring systems incorporate visual and audible alert mechanisms to inform drivers about abnormal operating conditions. Devices such as LCD displays, LED indicators, and buzzer alarms provide immediate feedback whenever system parameters exceed predefined safety limits. These features help drivers take corrective action before serious damage occurs.

Today, modern Smart Black Box Monitoring Systems combine multiple technologies such as embedded microcontrollers, real-time monitoring, event logging, communication protocols like I²C and UART, and alert mechanisms to improve vehicle safety and reliability. These systems continuously analyze important vehicle parameters and record critical events, making them valuable tools for diagnostics, maintenance, and accident analysis. By integrating

intelligent monitoring and event recording capabilities, smart black box systems contribute significantly to the advancement of modern electric vehicle safety and performance.

1.3 Advantages

1. Enhanced Vehicle Safety:

The system continuously monitors important vehicle parameters such as temperature, voltage, current, vibration, RPM, and battery level. If any parameter exceeds its safe threshold, the system immediately triggers alerts through LEDs and a buzzer, helping prevent potential vehicle failures or accidents.

2. Real-time Parameter Monitoring:

The Smart Black Box system constantly observes vehicle operating conditions and displays real-time parameter values on the LCD screen. This enables drivers or technicians to quickly understand the vehicle's current status and take necessary actions if abnormal conditions occur.

3. Accurate Event Logging with Time Stamp:

The integration of a Real Time Clock (RTC) through the I²C communication protocol allows the system to record the exact date and time whenever a fault or abnormal condition occurs. This event logging capability helps in analysing system performance, diagnosing faults, and maintaining proper vehicle records.

4. Simple and User-Friendly Interface:

The keypad interface allows users to easily select and monitor different vehicle parameters. The LCD display provides clear information about the selected parameter and system status, making the system easy to operate and understand.

5. Reliable Embedded Control and Communication:

The LPC1768 microcontroller, based on the ARM Cortex-M3 architecture, efficiently processes input data, performs threshold comparisons, and controls output devices. The system also uses I²C communication for RTC interfacing and UART for data logging, ensuring reliable communication between system components and external monitoring devices. Its high processing capability ensures fast response, reliable operation, and efficient system performance.

1.4 Problem Statement

Modern vehicles, especially electric vehicles, rely heavily on electronic systems for efficient operation and safety. However, many vehicles lack a dedicated system that continuously monitors critical parameters such as temperature, voltage, current, vibration,

motor speed, and battery level. Without proper monitoring, abnormal conditions may go unnoticed until they lead to system failures, reduced performance, or even vehicle accidents.

Existing vehicle monitoring systems often provide limited real-time diagnostics and do not record detailed event data when abnormal conditions occur. This lack of event logging makes it difficult to analyze faults, track vehicle performance, or investigate failures after they happen. Additionally, drivers may not receive immediate alerts when system parameters exceed safe operating limits, which increases the risk of equipment damage or unsafe driving conditions.

Another challenge is the absence of accurate time-based recording of faults and abnormal events. Without a proper timestamp, it becomes difficult to identify when a problem occurred or to analyse patterns of system behaviour over time. This limitation affects maintenance planning and fault diagnosis in modern vehicles.

To address these challenges, there is a need for a smart monitoring and event recording system that can continuously observe vehicle parameters, detect abnormal operating conditions, and alert the user in real time. Such a system should also be capable of logging important events with precise timestamps for further analysis.

This project aims to develop a Smart Black Box Monitoring and Event Logging System using the LPC1768 microcontroller, which continuously monitors key vehicle parameters, detects threshold violations, and provides visual and audible alerts. The system also records events with date and time information using a Real Time Clock (RTC) connected through the I2C communication protocol and transmits the data through UART communication. By integrating monitoring, alert mechanisms, event logging, and communication interfaces such as I2C and UART, the proposed system enhances vehicle safety, diagnostics, and operational reliability.

CHAPTER 2

LITERATURE OF SURVEY

2.1 Existing System

Traditional vehicle monitoring systems mainly rely on basic dashboard indicators to inform drivers about vehicle conditions. These systems provide only limited information such as engine temperature, battery status, or fuel level, without giving detailed data about the exact problem. As a result, drivers often become aware of faults only after vehicle performance is affected.

Early monitoring systems used simple sensors to detect parameters such as temperature and voltage. However, these systems usually do not record event data or store information about abnormal conditions. Once the vehicle is turned off, the recorded data is often lost, making it difficult to analyse faults or identify the cause of system failures.

Another major limitation of existing systems is the lack of real-time alerts and event logging. Most vehicles do not have a dedicated system to record parameters such as temperature, voltage, current, vibration, and motor speed along with the exact time of occurrence. This makes accident analysis and fault diagnosis more difficult.

Overall, the limitations of traditional vehicle monitoring systems highlight the need for an advanced smart monitoring system that can continuously observe vehicle parameters, provide alerts during abnormal conditions, and record important events for future analysis.

2.2 Need for New System

Despite advancements in vehicle technology, many traditional vehicle monitoring systems still have several limitations. Most vehicles rely on basic dashboard indicators that provide limited information about system performance. These indicators often fail to give detailed insights into important parameters such as temperature, voltage, current, vibration, and motor speed, making it difficult to detect problems at an early stage.

Another major issue in existing systems is the absence of proper event data recording mechanisms. When abnormal conditions occur, many vehicles do not store detailed information about the event, including the exact time and system parameters. This lack of recorded data makes fault diagnosis and accident analysis more challenging.

Real-time alert mechanisms are also limited in many conventional systems. Drivers may receive a warning signal, but they often cannot identify which specific parameter caused the

issue. Without proper monitoring and alert systems, small faults can develop into serious problems that affect vehicle safety and performance.

To overcome these limitations, there is a need for an intelligent monitoring system that can continuously observe important vehicle parameters and detect abnormal conditions. The proposed system integrates the LPC1768 microcontroller, keypad interface, Real Time Clock (RTC) using the I²C communication protocol, LCD display, LED indicators, buzzer alerts, and UART communication to monitor and record vehicle data effectively.

By combining real-time monitoring, alert mechanisms, and event logging with accurate timestamps, the proposed Smart Black Box Monitoring System provides a reliable solution for improving vehicle safety, diagnostics, and operational efficiency.

CHAPTER 3

ARM Cortex LPC1768

3.1 Introduction

The LPC1768 is a 32-bit microcontroller developed by NXP Semiconductors, based on the ARM Cortex-M3 core. It is designed for high-performance embedded applications, offering a balance of speed, efficiency, and extensive peripheral support. With a maximum clock speed of 100 MHz, it provides fast processing capabilities, making it suitable for real-time applications.

The microcontroller includes 512 KB of flash memory and 64 KB of SRAM, ensuring sufficient storage for program execution and data handling. It supports multiple communication protocols, including UART, I2C, SPI, CAN, and USB, allowing seamless integration with various devices. Additionally, the LPC1768 features a 12-bit ADC and DAC, PWM outputs, and GPIO pins, making it ideal for signal processing and control applications. Its integrated timers and watchdog functionality enhance system reliability, while low-power modes optimize energy consumption.

The ARM Cortex-M3 core enables efficient interrupt handling and multi-tasking, making it suitable for time-critical applications. Designed for embedded automation, IoT devices, and industrial control systems, the LPC1768 provides a robust solution for developers. Its ease of programming, supported by Keil, IAR, and GCC compilers, makes firmware development convenient.

The microcontroller supports real-time operating systems (RTOS), further enhancing its capability for multitasking environments. Its compatibility with the med platform enables quick prototyping and development of IoT applications. Featuring multiple power-saving modes, it extends battery life in portable applications. Its built-in Ethernet controller allows network connectivity for IoT and smart devices. The chip's flexible pin configuration offers scalability for different embedded designs. With high-speed general-purpose I/O (GPIO) and external memory interface support, it meets the requirements of complex applications.

3.2 Block Diagram of LPC1768

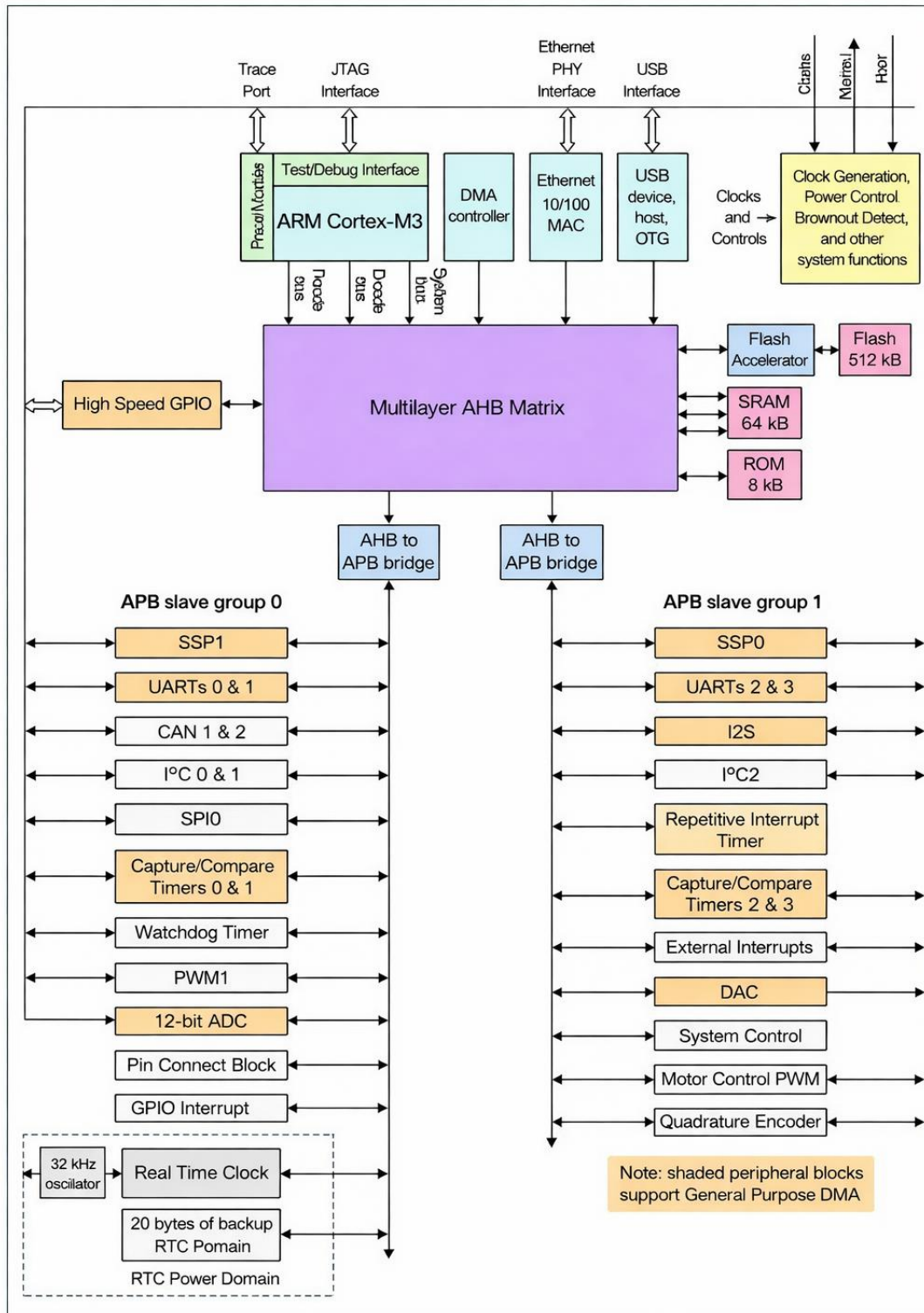


Fig 3.1: LPC1768 Block Diagram

3.3 Features

1. High-speed performance with low power consumption
2. Built-in ADC/DAC for sensor interfacing
3. Multiple communication interfaces for external devices
4. USB and Ethernet support for networking
5. Advanced interrupt handling and real-time execution

3.4 Pin Description

Pin No.	Pin Name	Primary Function	Alternate Functions
1	P1.18	GPIO	USB_UP_LED, PWM1.1, CAP1.0
2	P1.19	GPIO	MCOA0, USB_PPWR, CAP1.1
3	P1.20	GPIO	MCI0, PWM1.2, SCK0
4	P1.21	GPIO	MCI1, PWM1.3, SSEL0
5	P1.22	GPIO	MCI2, PWM1.4, MAT1.0
6	P1.23	GPIO	VBUS, PWM1.5, MAT1.1
7	P1.24	GPIO	SCK0, MCOB0, MAT0.0
8	P1.25	GPIO	MCOA1, PWM1.6, MAT0.1
9	P1.26	GPIO	MCOB1, STCLK, DCD1
10	P1.27	GPIO	CLKOUT, RI1, USB_OVRCCR
11	P1.28	GPIO	MCOA2, RTS1, CAP0.0
12	P1.29	GPIO	MCOB2, DSR1, MAT0.1
13	P1.30	GPIO	VBAT, DTR1, AD0.4
14	P1.31	GPIO	AD0.5, TXD3, VBUS
15	P0.0	GPIO	TXD3, SDA1
16	P0.1	GPIO	RXD3, SCL1
17	P0.2	GPIO	TXD0, AD0.7

18	P0.3	GPIO	RXD0, AD0.6
19	P0.4	GPIO	I2SRX_CLK, RD2
20	P0.5	GPIO	I2SRX_WS, TD2
21	P0.6	GPIO	I2SRX_SDA, SSEL1
22	P0.7	GPIO	I2STX_CLK, SCK1
23	P0.8	GPIO	I2STX_WS, MISO1
24	P0.9	GPIO	I2STX_SDA, MOSI1
25	P0.10	GPIO	TXD2, SDA2
26	P0.11	GPIO	RXD2, SCL2
27	P0.15	GPIO	TXD1, SCK0
28	P0.16	GPIO	RXD1, SSEL0
29	P0.17	GPIO	CTS1, MISO0
30	P0.18	GPIO	DCD1, MOSI0
31	P0.19	GPIO	DSR1, SCK1
32	P0.20	GPIO	DTR1, SSEL1
33	P0.21	GPIO	RI1, MAT3.0
34	P0.22	GPIO	RTS1, AD0.0
35	P0.23	GPIO	MAT3.1, AD0.1
36	P0.24	GPIO	MAT3.2, AD0.2
37	P0.25	GPIO	MAT3.3, AD0.3
38	P0.26	GPIO	AD0.4, AOUT
39	P0.27	GPIO	SDA0, CLKOUT
40	P0.28	GPIO	SCL0, USB_SCL

41	P0.29	GPIO	USB_D+, USB_DP
42	P0.30	GPIO	USB_D-, USB_DM
43	P0.31	GPIO	USB_UP_LED, USB_PPWR
44	P2.0	GPIO	PWM1.1, TXD1
45	P2.1	GPIO	PWM1.2, RXD1
46	P2.2	GPIO	PWM1.3, CTS1
47	P2.3	GPIO	PWM1.4, DCD1
48	P2.4	GPIO	PWM1.5, DSR1
49	P2.5	GPIO	PWM1.6, DTR1
50	P2.6	GPIO	PCAP1.0, RI1
51	P2.7	GPIO	RD2, USB_OVRCCR
52	P2.8	GPIO	TD2, ENET_MDC
53	P2.9	GPIO	USB_CONNECT, ENET_MDIO
54	P2.10	GPIO	EINT0, NMI
55	P2.11	GPIO	EINT1
56	P2.12	GPIO	EINT2
57	P2.13	GPIO	EINT3
58	P3.25	GPIO	MAT0.0
59	P3.26	GPIO	MAT0.1
60	P4.28	GPIO	RX_MCLK
61	P4.29	GPIO	TX_MCLK
62	P4.28	GPIO	RX_MCLK
63	P4.29	GPIO	TX_MCLK

64	P0.12	GPIO	USB_PPWR, AD0.6
65	P0.13	GPIO	USB_OVRCCR, AD0.7
66	P0.14	GPIO	EINT1, AD0.1
67	P0.15	GPIO	TXD1, SCK0
68	P0.16	GPIO	RXD1, SSEL0
69	P0.17	GPIO	CTS1, MISO0
70	P0.18	GPIO	DCD1, MOSI0
71	P0.19	GPIO	DSR1, SCK1
72	P0.20	GPIO	DTR1, SSEL1
73	P0.21	GPIO	RI1, MAT3.0
74	P0.22	GPIO	RTS1, AD0.0
75	P0.23	GPIO	MAT3.1, AD0.1
76	P0.24	GPIO	MAT3.2, AD0.2
77	P0.25	GPIO	MAT3.3, AD0.3
78	P0.26	GPIO	AD0.4, AOUT
79	P0.27	GPIO	SDA0, CLKOUT
80	P0.28	GPIO	SCL0, USB_SCL
81	P0.29	GPIO	USB_D+, USB_DP
82	P0.30	GPIO	USB_D-, USB_DM
83	P2.0	GPIO	PWM1.1, TXD1
84	P2.1	GPIO	PWM1.2, RXD1
85	P2.2	GPIO	PWM1.3, CTS1
86	P2.3	GPIO	PWM1.4, DCD1

87	P2.4	GPIO	PWM1.5, DSR1
88	P2.5	GPIO	PWM1.6, DTR1
89	P2.6	GPIO	PCAP1.0, RI1
90	P2.7	GPIO	RD2, USB_OVRCCR
91	P2.8	GPIO	TD2, ENET_MDC
92	P2.9	GPIO	USB_CONNECT, ENET_MDIO
93	P2.10	GPIO	EINT0, NMI
94	P2.11	GPIO	EINT1
95	P2.12	GPIO	EINT2
96	P2.13	GPIO	EINT3
97	VDD	Power (3.3V)	-
98	VSS	Ground	-
99	RESET	Reset	-
100	XTAL1	Crystal Oscillator	-

Table 3.1: Port and Pin Selection Table

3.5 LPC1768 Specification

Parameter	Specification
Core	ARM Cortex-M3
Clock Speed	100 MHz
Flash Memory	512 KB
SRAM	64 KB
I/O Pins	70 GPIOs
Communication Interfaces	UART, I2C, SPI, CAN, USB
ADC/DAC	12-bit ADC, 10-bit DAC

Table 3.2: Specifications of LPC1768 Microcontroller

Power Supply Specifications

1. **Operating Voltage:** 3.3V (core) & 5V (I/O compatibility).
2. **Minimum Supply Voltage:** 2.4V
3. **Maximum Supply Voltage:** 3.6V
4. **Typical Current Consumption:** 60 mA (depending on peripherals used).

3.6 Why LPC1768?

The ARM Cortex-M3-based LPC1768 microcontroller is chosen for the intruder detection system due to the following reasons:

1. **High Performance:** Operates at up to 100 MHz with a 32-bit ARM Cortex-M3 core.
2. **Low Power Consumption:** Supports various power-saving modes, making it efficient for continuous security monitoring.
3. **Multiple I/O Interfaces:** Includes GPIO, UART, I2C, SPI, and PWM, which are essential for connecting sensors, LCDs, buzzers, and keypads.
4. **Integrated Timers & Interrupts:** Helps in handling sensor input and alarm triggering with precision.
5. **Real-Time Processing:** Ensures quick response to intrusions.

3.7 Advantages And Disadvantages

Advantages:

1. High-speed operation (100 MHz) for real-time applications.
2. Rich peripheral support for interfacing multiple components.
3. Multiple power modes for energy efficiency.
4. On-chip Flash memory and SRAM for embedded applications.
5. Inbuilt ADC/DAC for analog interfacing.

Disadvantages:

1. More expensive compared to simpler microcontrollers.
2. Requires a complex programming setup.

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 Block Diagram and Schematic Diagram

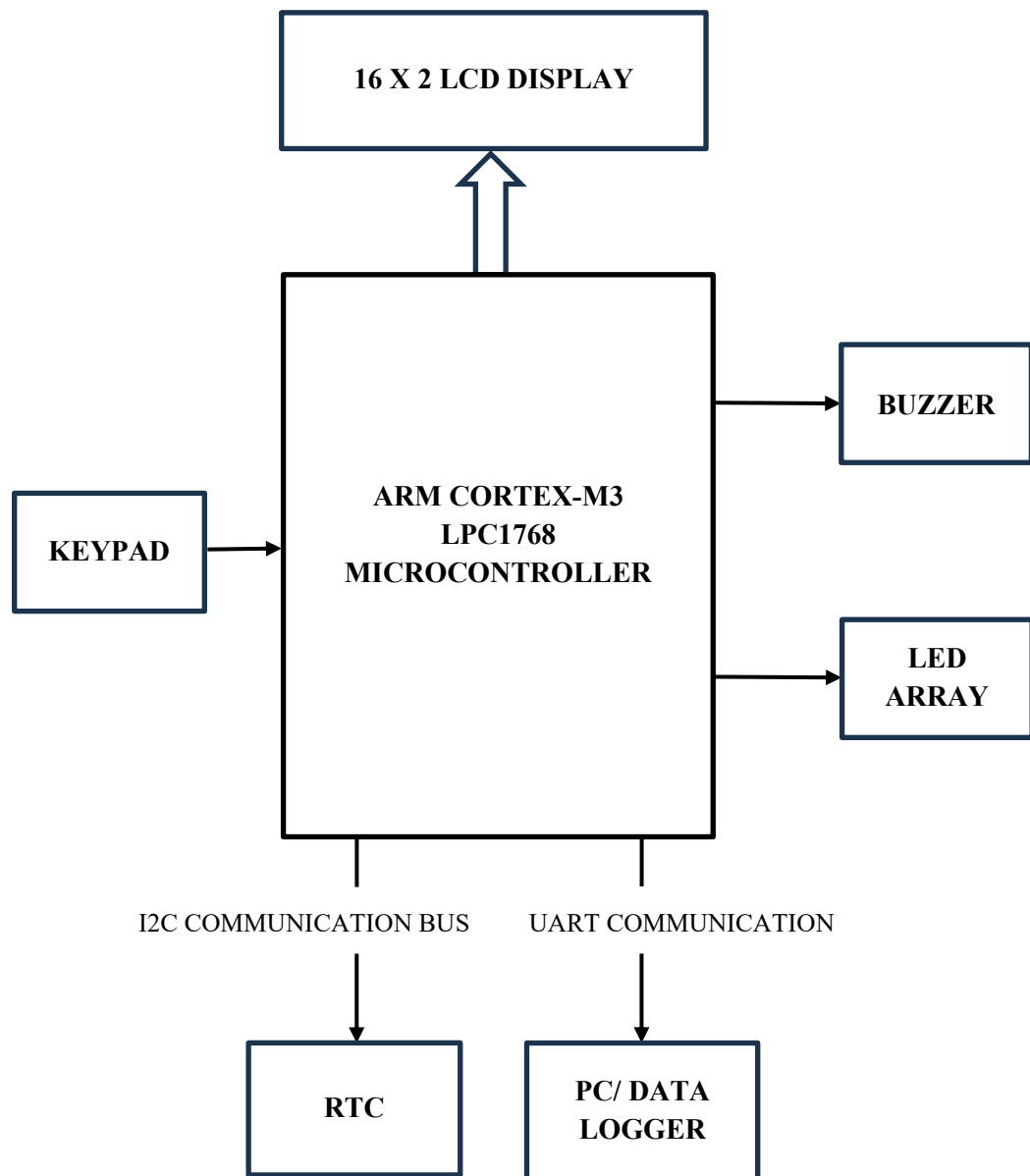


Fig 4.1: Block Diagram

SMART BLACK BOX – SCHEMATIC DIAGRAM

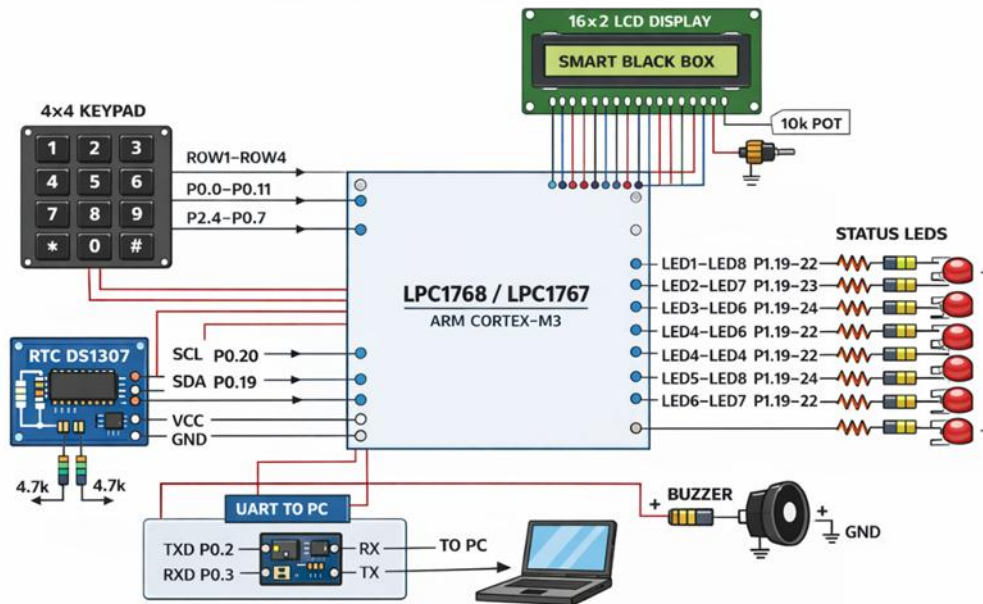


Fig 4.2: Schematic Diagram

The schematic diagram of the Smart Black Box represents the detailed electrical connections between all the hardware components used in the system. It shows how the LPC1767 ARM Cortex-M3 microcontroller interfaces with peripherals such as the 16×2 LCD display, 4×4 keypad, RTC (DS1307), LEDs, buzzer, and UART communication module. Each component is connected to specific GPIO pins of the controller. The LCD is used to display the selected parameter and its value, the keypad is used to select different parameters, and the RTC communicates through the I2C protocol to maintain time information. LEDs and the buzzer act as alert indicators when any parameter cross its threshold value.

The schematic diagram helps in understanding the exact wiring and signal flow between components before implementing the circuit on hardware or simulation software like Proteus. It includes details such as pull-up resistors for I2C communication, current limiting resistors for LEDs, and power connections for each module. By following the schematic diagram, the complete hardware circuit of the Smart Black Box can be assembled accurately, ensuring proper communication between the controller and peripheral devices.

4.2 Components and Connections

Component Specifications & Working Principles

1. Buzzer

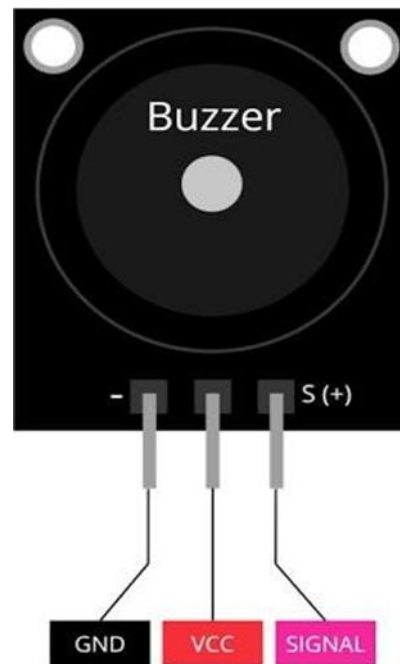


Fig 4.3 : Buzzer

Parameters & Specifications

Parameter	Specification
Operating Voltage	3V - 12V
Sound Level	85 dB
Type	Piezoelectric

Table 4.3: Specifications of Buzzer

Working Principle

When voltage is applied, the buzzer vibrates and generates sound to alert an intrusion.

Pin Description

Pin Description	Function
VCC	Power (3.3V/5V)
GND	Ground
IN	Signal input (from LPC1768)

Table 4.4: Pin description of Buzzer

2. LCD Display

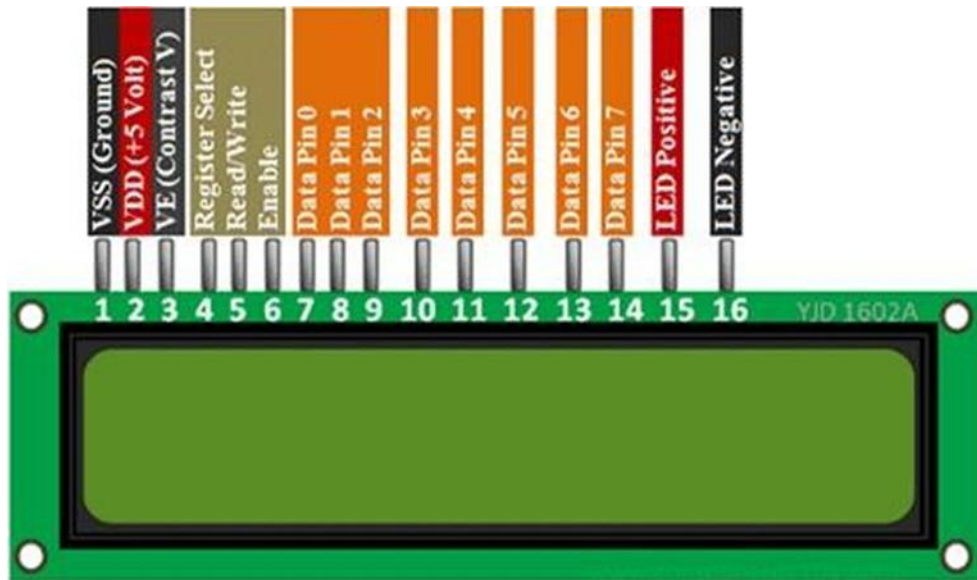


Fig 4.4: LCD Display

Parameters & Specifications

Parameter	Specification
Type	16x2 Character LCD
Operating Voltage	5V
Interface	I2C/SPI/Parallel

Table 4.5: Specifications of LCD display

Working Principle

The LCD displays characters using liquid crystal technology. Data is sent from the LPC1768 via data pins. and control signals from control.

Pin Description

Pin Description	Function
VCC	5V Power Supply
GND	Ground
RS, RW, E	Control Pins
D0-D7	Data Pins

Table 4.6: Pin description of LCD display

3. LED INDICATORS



Fig 4.5: Array of LED's

Parameters & Specifications

Parameter	Specification
Type	LED Indicators
Operating Voltage	3.3V-5V
Interface	GPIO
Color	Green
Purpose	Status / Alert indication

Table 4.7: Specifications of LED indicators

Working Principle

The LED indicators are connected to the GPIO pins of the LPC1768 microcontroller and provide visual status indications. When a monitored parameter exceeds its threshold value, the microcontroller activates the corresponding LED to alert the user.

Pin Description

Pin Description	Function
VCC	Power supply for LED module
GND	Ground connection
P1.18	LED1 control
P1.19	LED2 control
P1.20	LED3 control
P1.21	LED4 control
P1.22	LED5 control
P1.23	LED6 control
P1.24	LED7 control
P1.25	LED8 control

Table 4.8: Pin description of LED indicators

4. KEYPAD MODULE



Fig 4.6: 4×4 Keypad Module

Parameters & Specifications

Parameter	Specification
Type	4×4 Matrix Keypad
Number of Keys	16 Keys
Operating Voltage	3.3V – 5V
Interface	GPIO (Matrix Interface)
Rows	4
Columns	4

Table 4.9: Specifications of 4×4 Keypad Module

Working Principle

The 4×4 keypad works on a matrix arrangement of rows and columns. When a key is pressed, it connects a specific row and column, allowing the LPC1768 microcontroller to detect the pressed key through scanning.

Pin Description

Pin	Function
Row 1	Row input/output
Row 2	Row input/output
Row 3	Row input/output
Row 4	Row input/output
Column 1	Column input
Column 2	Column input
Column 3	Column input
Column 4	Column input

Table 4.10: Pin description of 4×4 Keypad

5. REAL TIME CLOCK

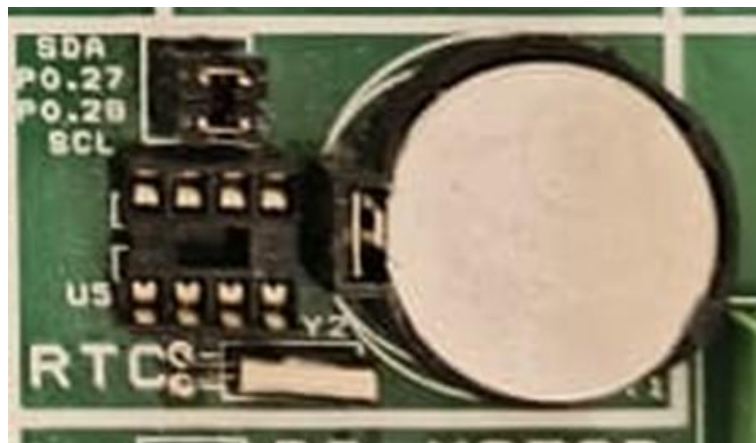


Fig 4.7: Real Time Clock Module

Parameters & Specifications

Parameter	Specification
Type	DS1307 / DS3231 RTC Module
Operating Voltage	3.3V – 5V
Communication	I ² C Protocol
Time Format	12 / 24 Hour
Function	Real-time date and time tracking

Table 4.11: Specifications of RTC Module

Working Principle

The RTC module keeps track of current date and time using an internal oscillator and backup battery. It communicates with the LPC1768 microcontroller through the I²C communication protocol to provide accurate time information for event logging.

Pin Description

Pin	Function
VCC	Power Supply
GND	Ground
SDA	Serial Data Line (I ² C)
SCL	Serial Clock Line (I ² C)

Table 4.12: Pin Description of RTC Module

6. URAT



Fig 4.7: UART Communication Interface

Parameters & Specifications

Parameter	Specification
Type	Serial Communication Interface
Communication Protocol	UART
Baud Rate	9600 bps
Data Bits	8
Stop Bits	1

Table 4.13: Specifications of UART Communication

Working Principle

UART communication allows the LPC1768 microcontroller to transmit and receive serial data. In this system, UART is used to send parameter values and event logs to a PC for monitoring and analysis.

Pin Description

Pin	Function
TXD	Transmit Data
RXD	Receive Data
GND	Ground Reference

Table 4.14: Pin description of UART interface

Circuit Connections

Component	LPC1768 Pin
4×4 Keypad (Rows)	P2.0 – P2.3
4×4 Keypad (Columns)	P2.4 – P2.7
LED Indicators	P1.18 – P1.25
Buzzer	P1.27
LCD Display (16×2) Data Pins	P0.15 – P0.22
LCD Display Control Pins	RS → P0.10, EN → P0.11
RTC Module (I ² C SDA)	P0.27
RTC Module (I ² C SCL)	P0.28
UART TX	P0.2
UART RX	P0.3

Table 4.15: Circuit connections

4.4 Working mechanism of Smart Black Box Monitoring System

1. Parameter Selection and Monitoring

1. The system uses a 4×4 keypad to allow the user to select different vehicle parameters such as temperature, voltage, current, vibration, RPM, and battery level.
2. When a key is pressed, the LPC1768 microcontroller reads the selected parameter and begins monitoring its value.
3. The corresponding parameter value is generated or received and then displayed on the 16×2 LCD display.
4. The system continuously checks the parameter value against predefined threshold limits to detect abnormal conditions.

2. Event Detection and Alert Mechanism

1. The LPC1768 microcontroller compares the monitored parameter value with its safe threshold level.
2. If the parameter value remains within the normal range, the system displays “Normal” on the LCD display.
3. If the parameter value exceeds the safe limit, the system:
 - a. Activates the corresponding LED indicator to show which parameter triggered the alert.
 - b. Turns on the buzzer to generate an audible warning signal.
4. This alert mechanism helps users quickly identify abnormal conditions and take corrective action.

3. Event Logging and Communication

1. The system uses a Real Time Clock (RTC) module connected through the I²C communication protocol to maintain accurate date and time information.
2. Whenever an abnormal condition occurs, the system:
 - a. Records the parameter name and value.
 - b. Stores the exact date and time of the event using the RTC.
3. The event data is then transmitted through UART communication to a connected computer or monitoring device.
4. This data logging feature allows technicians or users to analyze vehicle performance, detect faults, and maintain system records.

CHAPTER 5

5.1 Keil uVision IDE

The Keil μ Vision IDE is a powerful development environment used for programming and debugging embedded systems based on microcontrollers such as the LPC1768. It provides a complete platform for writing, compiling, and testing programs using Embedded C language. The IDE integrates several essential tools including a code editor, compiler, linker, debugger, and simulator, making it highly suitable for developing embedded applications.

In this project, the Keil μ Vision IDE is used to write and compile the program that controls the Smart Black Box Monitoring and Event Logging System. The IDE supports syntax highlighting and error detection features that help programmers write code efficiently and identify errors during the development process

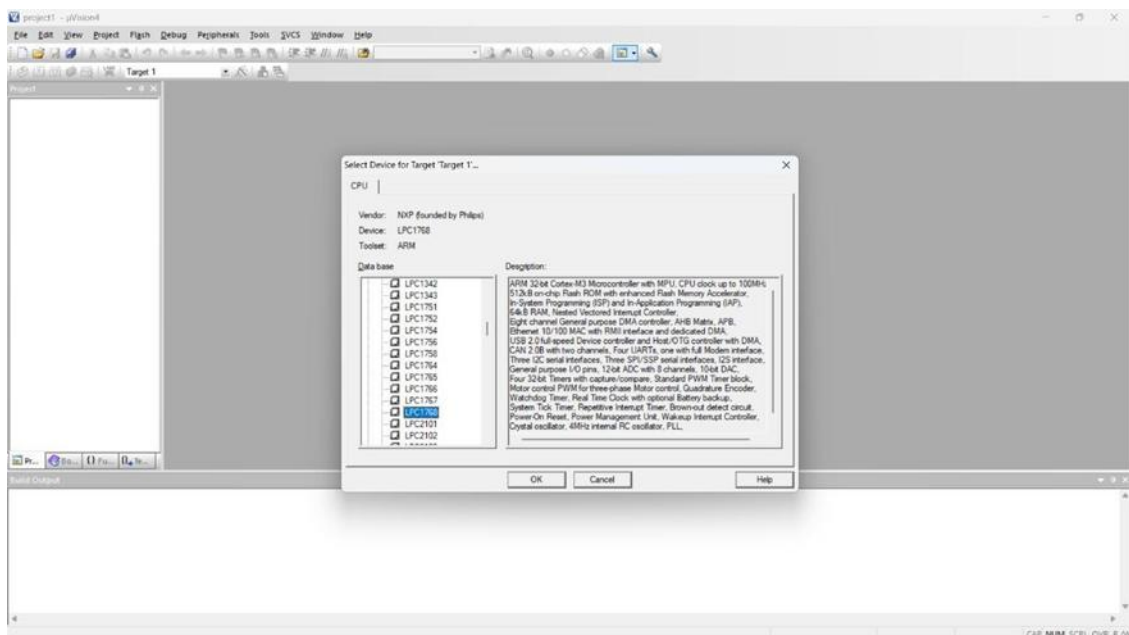


Fig 5.1: Selecting LPC1768 as the Target

To create a new project in Keil μ Vision, the user must first open the IDE and select Project $\rightarrow$ New Project. After creating a project folder, the user must choose the target microcontroller (LPC1768) from the device database provided by the IDE. This step configures the project for the specific microcontroller architecture, as shown in the Fig 5.1.

After selecting the microcontroller, the source code file is added to the project and the program is written using the built-in code editor. The editor provides useful features such as syntax highlighting, automatic indentation, and error detection to simplify coding.

Once the program is written, the user can compile the code using the Build option available in the IDE. The compiler checks the program for syntax errors and generates the

corresponding hex or binary file required for the microcontroller. This process is illustrated in Fig 5.2

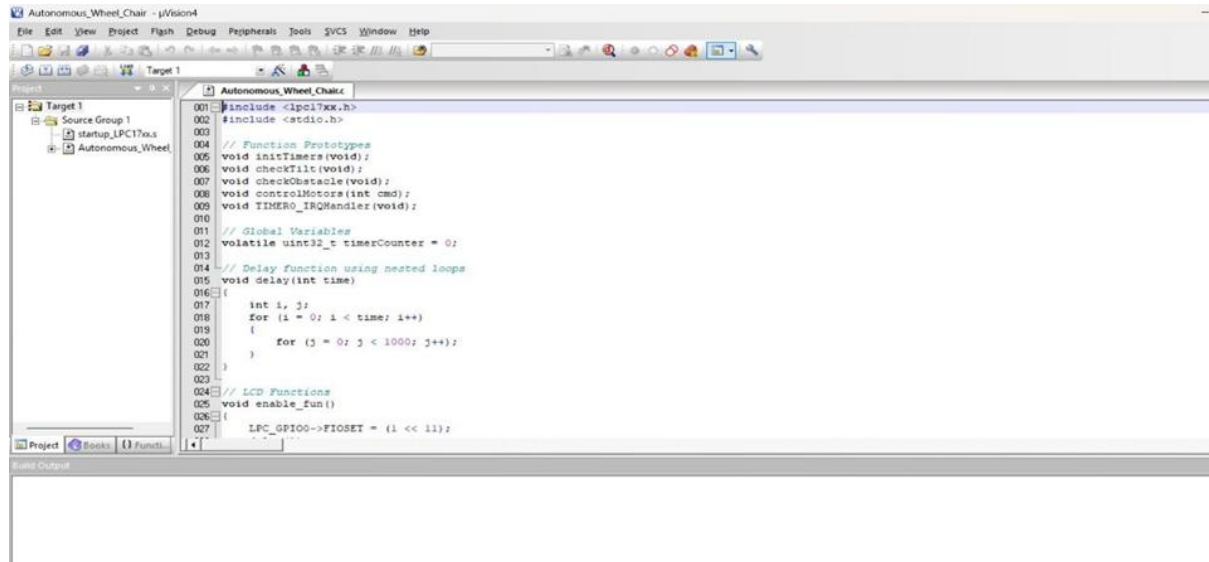


Fig 5.2: Writing and Building the code to the target

The Keil μ Vision IDE also provides an integrated debugger that allows the user to test and analyze the program step-by-step. The debugger enables developers to set breakpoints, monitor variables, and observe program execution during runtime. This feature is very useful for identifying logical errors and improving program reliability.

Additionally, the IDE includes a simulation environment that allows developers to test the code without immediately loading it onto the hardware. By using the simulator, the user can verify the program behavior before downloading it to the LPC1768 development board, thereby reducing development time and improving efficiency.

5.2 Embedded C Programming

The Smart Black Box Monitoring and Event Logging System is designed to enhance vehicle safety by continuously monitoring important vehicle parameters and recording abnormal events. The system ensures reliable operation by detecting unsafe conditions and alerting the user through visual and audible signals.

The firmware for the Smart Black Box Monitoring System is written in Embedded C, a specialized version of the C programming language designed for embedded systems. Embedded C is suitable for this project because it provides efficient hardware control, fast execution, and real-time responsiveness. The program is divided into multiple functions, each responsible for performing specific tasks such as keypad reading, LCD display control, RTC communication through I²C, parameter monitoring, and UART data transmission.

The main function initializes the system by configuring the GPIO pins for the keypad, LEDs, and buzzer, setting up the LCD display for real-time parameter visualization, initializing the I²C communication for the RTC module, and configuring UART communication for

sending data to an external computer. Once initialization is completed, the program enters a continuous loop where the system constantly monitors selected parameters.

When the user presses a key on the keypad, the microcontroller identifies the selected parameter such as temperature, voltage, current, vibration, RPM, or battery level. The system generates or reads the corresponding parameter value and displays it on the 16×2 LCD display. At the same time, the program checks whether the parameter value exceeds its predefined safety threshold.

The program uses conditional statements to detect abnormal operating conditions. If the parameter value remains within the safe range, the LCD displays the system status as “Normal.” However, if the parameter exceeds the threshold value, the system activates the corresponding LED indicator and buzzer alert, warning the user about the abnormal condition.

To ensure accurate event recording, the system uses a Real Time Clock (RTC) module connected through the I²C communication protocol. Whenever an alert condition occurs, the microcontroller retrieves the current date and time from the RTC and attaches the timestamp to the event log.

The system also utilizes UART communication to transmit monitored parameter values and event logs to a computer or external monitoring system. This allows technicians or users to store and analyze system data for diagnostics and maintenance purposes.

Overall, the Embedded C program for the Smart Black Box Monitoring System is designed to be efficient, responsive, and reliable. It ensures real-time monitoring, automatic fault detection, accurate event logging, and clear alert mechanisms, thereby improving vehicle safety and system diagnostics.

5.3 GPIO Interfacing

The Smart Black Box Monitoring System utilizes the GPIO peripherals of the LPC1768 microcontroller to interface with various input and output devices. The GPIO pins are configured as either inputs or outputs depending on their function in the system. The keypad (P2.0 – P2.7) is configured as an input device that allows the user to select different vehicle parameters such as temperature, voltage, current, vibration, RPM, and battery level for monitoring.

The LED indicators (P1.18 – P1.25) are configured as output pins and provide visual alerts whenever a monitored parameter exceeds its predefined threshold value. The buzzer (P1.27) is also connected as an output device and generates an audible alert to warn the user about abnormal operating conditions.

The LCD display (16×2) is connected to the LPC1768 through data pins (P0.15 – P0.22) and control pins (P0.10 and P0.11) to display real-time parameter values and system status messages. The system also includes a Real Time Clock (RTC) module connected via the I²C

communication interface (SDA – P0.27, SCL – P0.28) to maintain accurate date and time for event logging. Additionally, UART communication (TX – P0.2, RX – P0.3) is used to transmit monitored data and alert information to an external computer for logging and analysis.

5.4 Flash Magic

Flash Magic is a tool used for programming and debugging microcontrollers, including the LPC1768. It is particularly useful for flashing the HEX file generated by the Keil uVision IDE onto the microcontroller. Flash Magic supports various communication interfaces, such as UART, and provides a user-friendly interface for configuring the microcontroller's settings and flashing the code. The detailed interface was shown in the Fig 5.3.

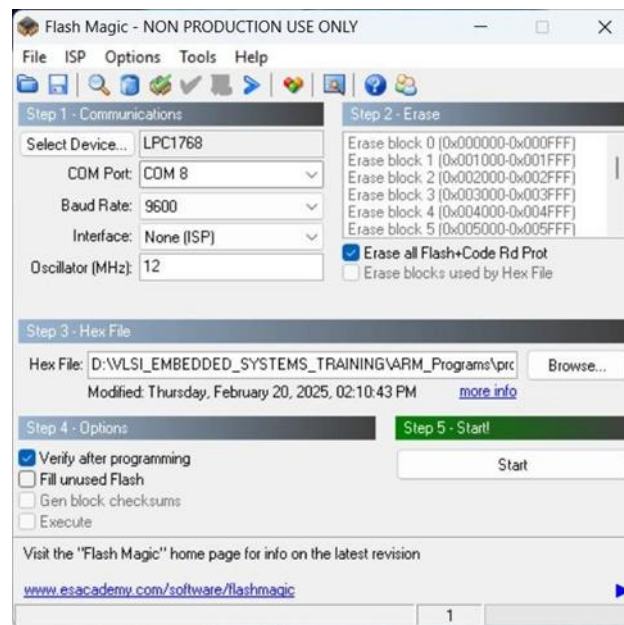


Fig 5.3: Dumping the code using Flash Magic

5.5 Code for LPC1768

Source code

```
#include <LPC17xx.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ----- LCD ----- */
#define LCD_RS (1<<10)
#define LCD_EN (1<<11)
#define LCD_DATA (0xFF<<15)

/* ----- BUZZER & LED ----- */
#define BUZZER (1<<27)
#define LED_START 19
#define LED_MASK (0xFF<<LED_START)

/* ----- I2C RTC ----- */
#define RTC_ADDR 0x68

/* ----- DELAY ----- */
void delay_ms(uint32_t ms)
{
    uint32_t i,j;
    for(i=0;i<ms;i++)
        for(j=0;j<10000;j++);
}

/* ----- LCD ----- */
void lcd_enable()
{
    LPC_GPIO0->FIOSET = LCD_EN;
    delay_ms(2);
    LPC_GPIO0->FIOCLR = LCD_EN;
}

void lcd_cmd(unsigned char cmd)
{
    LPC_GPIO0->FIOCLR = LCD_RS | LCD_DATA;
    LPC_GPIO0->FIOSET = (cmd<<15);
    lcd_enable();
}

void lcd_data(unsigned char data)
{
    LPC_GPIO0->FIOSET = LCD_RS;
    LPC_GPIO0->FIOCLR = LCD_DATA;
    LPC_GPIO0->FIOSET = (data<<15);
    lcd_enable();
}

void lcd_init()
{
    LPC_GPIO0->FIODIR |= LCD_RS | LCD_EN | LCD_DATA;

    lcd_cmd(0x38);
    lcd_cmd(0x0C);
```

```

    lcd_cmd(0x06);
    lcd_cmd(0x01);
}

void lcd_str(char *s)
{
    while(*s) lcd_data(*s++);
}

/* ----- UART ----- */
void uart_init()
{
    LPC_SC->PCONP |= (1<<3);

    LPC_PINCON->PINSEL0 |= (1<<4);
    LPC_PINCON->PINSEL0 |= (1<<6);

    LPC_UART0->LCR = 0x83;
    LPC_UART0->DLL = 6;
    LPC_UART0->DLM = 0;
    LPC_UART0->FCR = 0x07;
    LPC_UART0->LCR = 0x03;
}

void uart_char(char c)
{
    while(!(LPC_UART0->LSR & (1<<5)));
    LPC_UART0->THR = c;
}

void uart_str(char *s)
{
    while(*s) uart_char(*s++);
}

/* ----- I2C ----- */
void i2c_init()
{
    LPC_SC->PCONP |= (1<<7);

    LPC_PINCON->PINSEL1 |= (1<<22);
    LPC_PINCON->PINSEL1 |= (1<<24);

    LPC_I2C1->SCLH = 60;
    LPC_I2C1->SCLL = 60;

    LPC_I2C1->I2CONSET = (1<<6);
}

void i2c_start()
{
    LPC_I2C1->I2CONSET = (1<<5);
    while(!(LPC_I2C1->I2CONSET & (1<<3)));
}

void i2c_stop()
{
    LPC_I2C1->I2CONSET = (1<<4);
    LPC_I2C1->I2CONCLR = (1<<5);
}

```

```

}

void i2c_write(char data)
{
    LPC_I2C1->I2DAT = data;
    LPC_I2C1->I2CONCLR = (1<<3);
    while(!(LPC_I2C1->I2CONSET & (1<<3)));
}

/* ----- KEYPAD ----- */
const char keypad[4][4]={
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

char keypad_read()
{
    int r,c;

    LPC_GPIO2->FIOSET = 0x0F;

    for(r=0;r<4;r++)
    {
        LPC_GPIO2->FIOCLR = (1<<r);

        for(c=0;c<4;c++)
        {
            if(!(LPC_GPIO2->FIOPIN & (1<<(c+4))))
            {
                while(!(LPC_GPIO2->FIOPIN & (1<<(c+4))));
                LPC_GPIO2->FIOSET = (1<<r);
                return keypad[r][c];
            }
        }

        LPC_GPIO2->FIOSET = (1<<r);
    }

    return 0;
}

/* ----- PARAMETER NAME ----- */
void get_name(char key,char *name)
{
    switch(key)
    {
    case '1': strcpy(name,"Temp"); break;
    case '2': strcpy(name,"Pressure"); break;
    case '3': strcpy(name,"Humidity"); break;
    case '4': strcpy(name,"Voltage"); break;
    case '5': strcpy(name,"Current"); break;
    case '6': strcpy(name,"Vibration"); break;
    case '7': strcpy(name,"RPM"); break;
    case '8': strcpy(name,"Fuel"); break;
    }
}

```

```

/* ----- RANDOM VALUE ----- */
int generate_value(char key)
{
    switch(key)
    {
        case '1': return rand()%60+20;
        case '2': return rand()%60+950;
        case '3': return rand()%80+20;
        case '4': return rand()%30+210;
        case '5': return rand()%15+3;
        case '6': return rand()%6+1;
        case '7': return rand()%4000+800;
        case '8': return rand()%100;
    }
    return 0;
}

/* ----- THRESHOLD ----- */
int check_threshold(char key,int value)
{
    switch(key)
    {
        case '1': return value>50;
        case '2': return value>990;
        case '3': return value>70;
        case '4': return value>235;
        case '5': return value>12;
        case '6': return value>4;
        case '7': return value>3500;
        case '8': return value<20;
    }
    return 0;
}

/* ----- MAIN ----- */
int main()
{
    char key;
    char pname[15];
    char buffer[100];
    int led_pin;

    LPC_GPIO2->FIODIR |= 0x0F;
    LPC_GPIO2->FIODIR &= ~(0xF0);
    LPC_GPIO1->FIODIR |= LED_MASK | BUZZER;

    lcd_init();
    uart_init();
    i2c_init();

    lcd_str("SMART BLACK BOX");

    while(1)
    {
        key = keypad_read();

        if(key>='1' && key<='8')
        {
            int value = generate_value(key);

```

```

int alert = check_threshold(key,value);

get_name(key,pname);

lcd_cmd(0x01);
lcd_str(pname);
lcd_str(":");

sprintf(buffer,"%d",value);
lcd_str(buffer);

led_pin = (key-'1')+LED_START;

LPC_GPIO1->FIOCLR = LED_MASK | BUZZER;

lcd_cmd(0xC0);

if(alert)
{
    LPC_GPIO1->FIOSET = (1<< led_pin) | BUZZER;

    lcd_str("ALERT");
}
else
{
    lcd_str("NORMAL");
}

sprintf(buffer,"\r\n%s:%d %s\r\n",pname,value,alert?"ALERT":"OK");

uart_str(buffer);

delay_ms(500);
}
}
}

```


CHAPTER 6

RESULTS AND OBSERVATIONS

6.1 Parameter Selection Using Keypad

The Smart Black Box Monitoring System uses a 4×4 keypad to select different vehicle parameters for monitoring. Each key corresponds to a specific parameter. When a parameter is selected, the system reads the value through sensors or ADC inputs and displays it on the LCD display.

If the parameter value remains within the safe range, the system displays “Normal”. If the value exceeds the threshold limit, the system triggers an alert condition, where the buzzer activates and the corresponding LED blinks.

Parameter Selection Keys

Key	Parameter
1	Temperature
2	Pressure
3	Humidity
4	Voltage
5	Current
6	Vibration
7	RPM
8	Fuel Level

6.2 System Initialization

Description:

When the system is powered ON, the LPC1768 initializes all peripherals including LCD, UART, RTC, ADC, LEDs, and buzzer.

System Response

- LCD Display: “Black Box Ready”
- RTC starts tracking date and time
- System enters monitoring mode

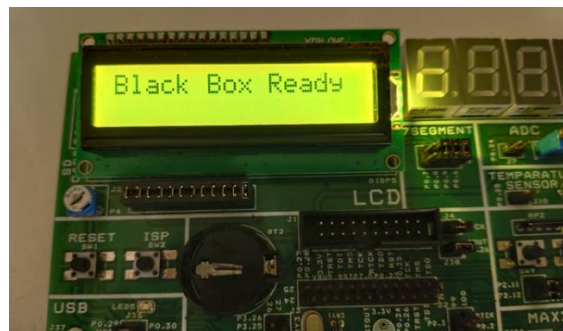


Fig 6.1: System Initialization Display

6.3 Temperature Monitoring

Description:

The system monitors the temperature value using the temperature sensor connected to the ADC channel.

Sensor Reading

Temperature = 48°C

System Response

- LCD Display: Temp:48
- Status: OK
- LED: OFF
- Buzzer: OFF

The temperature remains within the safe operating range.

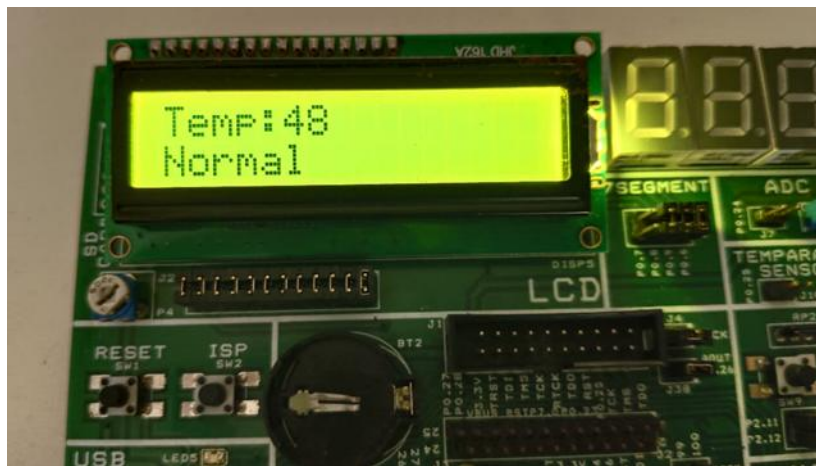


Fig 6.2: Temperature Monitoring Display

6.4 Pressure Monitoring

Description:

The pressure parameter is monitored to ensure that the system operates within safe limits.

Normal Condition

Sensor Reading

Pressure = 977

System Response

- LCD Display: Pressure:977 Normal
- LED: OFF
- Buzzer: OFF



Fig 6.3: Pressure Normal Condition

Alert Condition

Sensor Reading

Pressure = **999**

System Response

- LCD Display: Pressure:999 ALERT
- LED: Blinking
- Buzzer: Activated

This indicates the pressure has crossed the predefined threshold.

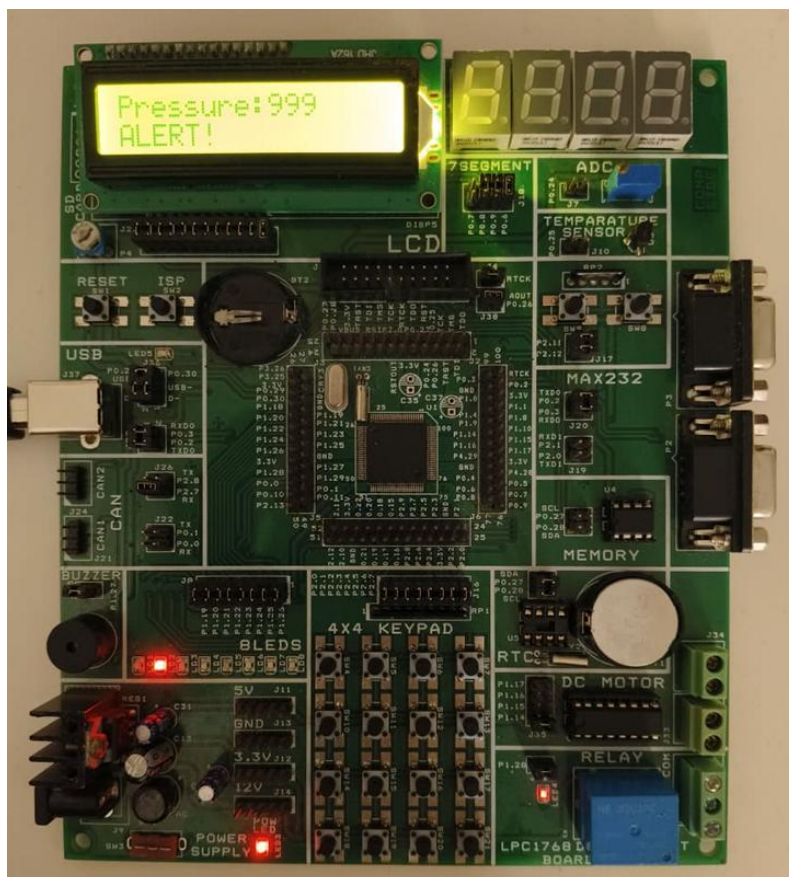


Fig 6.4: Pressure Alert Condition

6.5 Humidity Monitoring

Description:

Humidity level is monitored as part of environmental conditions affecting the vehicle system.

Sensor Reading

Humidity = 43

System Response

- LCD Display: Humidity:43 Normal
- LED: OFF
- Buzzer: OFF

The humidity value remains within the acceptable range.



Fig 6.5: Humidity Monitoring Display

6.6 Voltage Monitoring

Voltage monitoring is essential to detect abnormal power conditions in the vehicle.

Normal Condition

Sensor Reading

Voltage = 221 V

System Response

- LCD Display: Voltage:221 Normal
- LED: OFF
- Buzzer: OFF

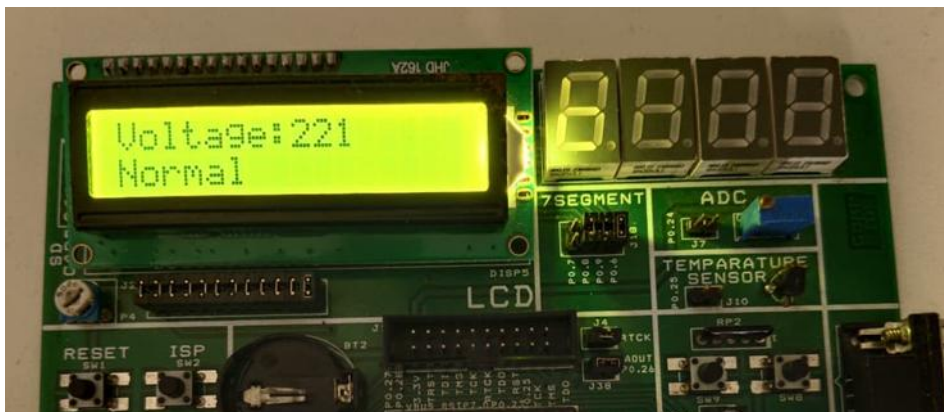


Fig 6.6: Voltage Normal Condition

Alert Condition

Sensor Reading

Voltage = 237 V

System Response

- LCD Display: Voltage:237 ALERT
- LED: Blinking
- Buzzer: Activated

This indicates that the voltage has exceeded the safe threshold.

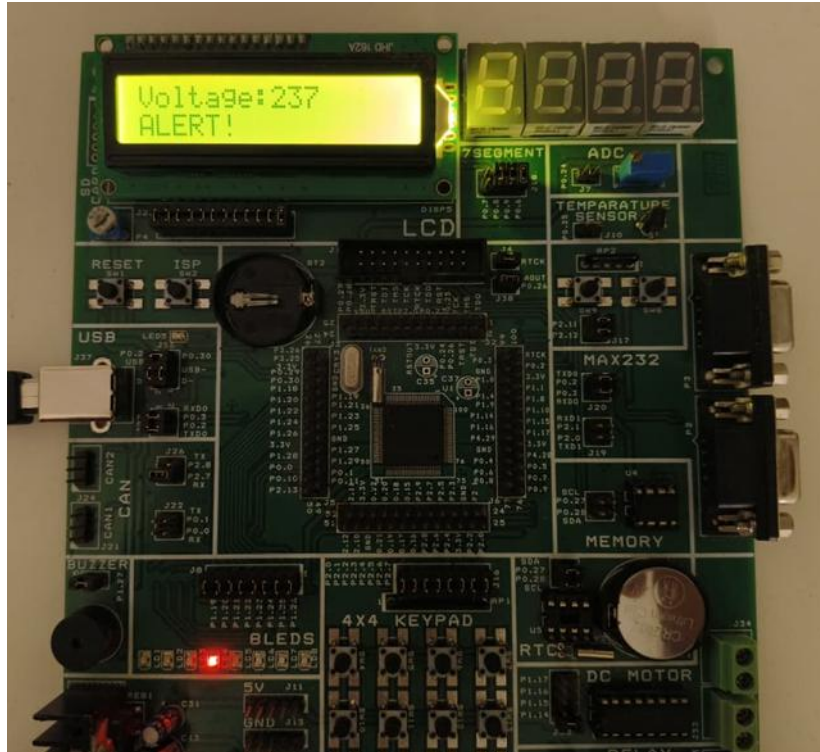


Fig 6.7: Voltage Alert Condition

6.7 Current Monitoring

Sensor Reading

Current = 6 A

System Response

- LCD Display: Current:6 Normal
- LED: OFF
- Buzzer: OFF

The system confirms normal current flow.

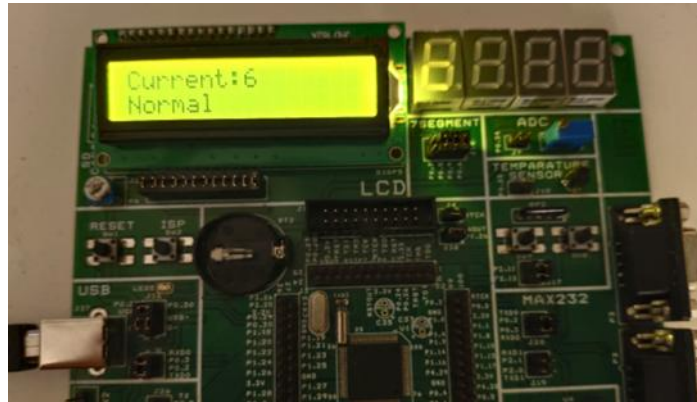


Fig 6.8: Current Monitoring Display

6.8 Vibration Monitoring

Sensor Reading

Vibration = 2

System Response

- LCD Display: Vibration:2 Normal
- LED: OFF
- Buzzer: OFF

The vibration level remains within acceptable limits.



Fig 6.9: Vibration Monitoring Display

6.9 RPM Monitoring

Sensor Reading

RPM = 2874

System Response

- LCD Display: **RPM:2874 Normal**
- LED: OFF
- Buzzer: OFF

The motor speed is within the normal operating range.

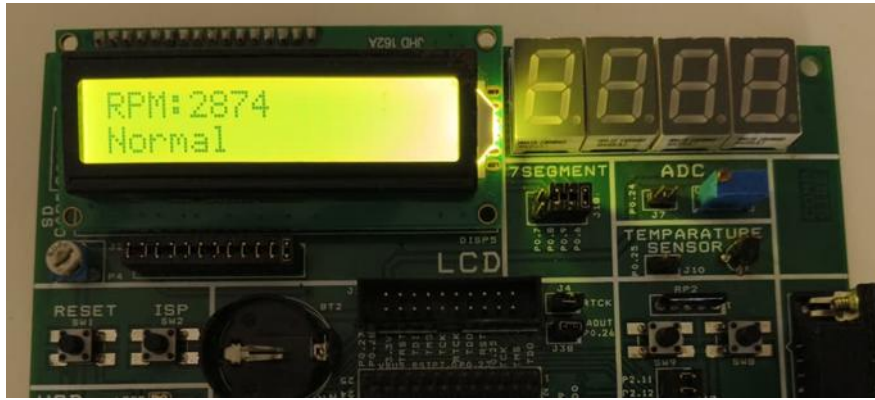


Fig 6.10: RPM Monitoring Display

6.10 Fuel Level Monitoring

Sensor Reading

Fuel Level = 73%

System Response

- LCD Display: **Fuel:73 Normal**
- LED: OFF
- Buzzer: OFF

The fuel level is sufficient for normal operation.

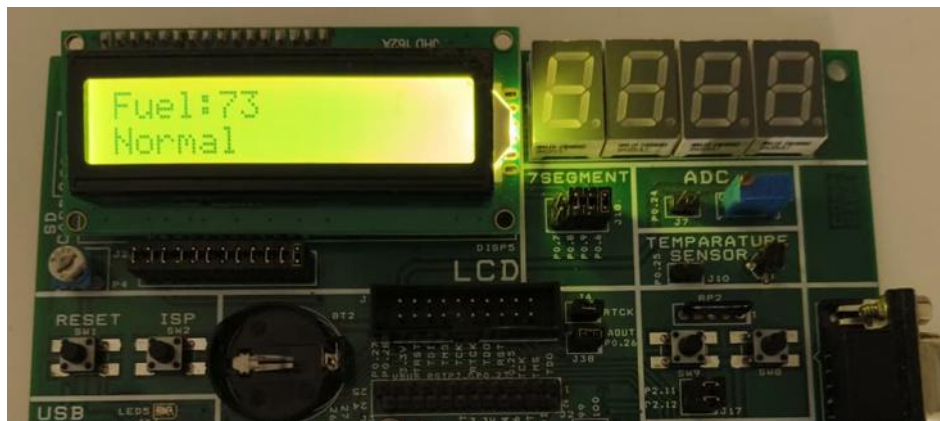


Fig 6.11: Fuel Level Monitoring Display

6.11 UART Event Logging

The system also transmits all monitored parameter values to the UART terminal along with date and time information obtained from the RTC module.

Each event includes:

- Parameter value
- Date
- Time
- System status (OK / ALERT)

Example output displayed in the UART terminal:

Temp:48 | Date:12/02/2026 Time: 14:00:05 OK

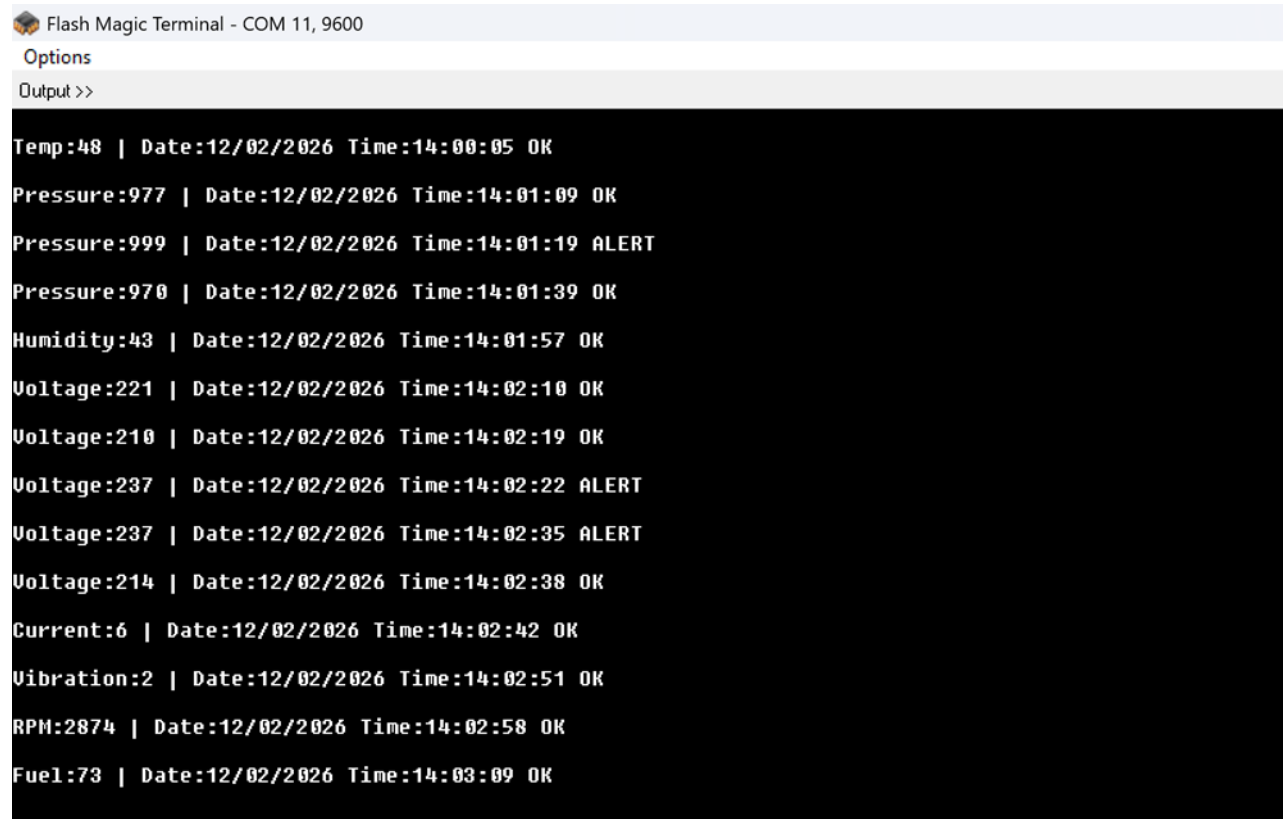
Pressure:999 | Date:12/02/2026 Time: 14:01:19 ALERT

Voltage:237 | Date:12/02/2026 Time: 14:02:22 ALERT

RPM:2874 | Date:12/02/2026 Time: 14:02:58 OK

Fuel:73 | Date:12/02/2026 Time: 14:03:09 OK

This event logging mechanism helps in fault analysis, diagnostics, and maintenance tracking.

A screenshot of a terminal window titled "Flash Magic Terminal - COM 11, 9600". The window has a menu bar with "Options" and a status bar with "Output >>". The main area displays a log of sensor data events. Each line follows the format: "SensorName:Value | Date:12/02/2026 Time:HH:MM:SS Status". The sensors include Temp, Pressure, Humidity, Voltage, Current, Vibration, RPM, and Fuel. The status is either "OK" or "ALERT".

```
Temp:48 | Date:12/02/2026 Time:14:00:05 OK
Pressure:977 | Date:12/02/2026 Time:14:01:09 OK
Pressure:999 | Date:12/02/2026 Time:14:01:19 ALERT
Pressure:970 | Date:12/02/2026 Time:14:01:39 OK
Humidity:43 | Date:12/02/2026 Time:14:01:57 OK
Voltage:221 | Date:12/02/2026 Time:14:02:10 OK
Voltage:210 | Date:12/02/2026 Time:14:02:19 OK
Voltage:237 | Date:12/02/2026 Time:14:02:22 ALERT
Voltage:237 | Date:12/02/2026 Time:14:02:35 ALERT
Voltage:214 | Date:12/02/2026 Time:14:02:38 OK
Current:6 | Date:12/02/2026 Time:14:02:42 OK
Vibration:2 | Date:12/02/2026 Time:14:02:51 OK
RPM:2874 | Date:12/02/2026 Time:14:02:58 OK
Fuel:73 | Date:12/02/2026 Time:14:03:09 OK
```

Fig 6.12: UART Event Logging Output

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The Smart Black Box Monitoring and Event Logging System for Electric Vehicles was successfully designed and implemented to improve vehicle safety and monitoring capabilities. The system continuously observes critical vehicle parameters and records important events whenever abnormal conditions occur. By integrating hardware components such as the LPC1768 microcontroller, keypad, LCD display, LED indicators, buzzer, RTC module, and communication interfaces, the system provides an efficient platform for monitoring and analyzing vehicle performance.

The LPC1768 microcontroller, based on the ARM Cortex-M3 architecture, serves as the central controller of the system. It performs all major operations such as reading user input from the keypad, processing parameter values, comparing them with predefined threshold limits, and controlling the output devices. The system monitors important parameters including temperature, voltage, current, vibration, motor speed (RPM), and battery level, which represent the overall health and operational condition of an electric vehicle.

Whenever a monitored parameter exceeds its safe threshold value, the system immediately generates visual alerts through LED indicators and audible alerts through a buzzer. This helps drivers or technicians quickly identify abnormal conditions and take corrective actions before serious damage occurs. The LCD display provides real-time information about the selected parameter and system status, making the system simple and user-friendly.

A key feature of the system is its event logging capability. The integration of a Real Time Clock (RTC) module using the I²C communication protocol allows the system to record the exact date and time when an abnormal condition occurs. This timestamped data enables accurate tracking of faults and helps in analyzing system performance over time. Additionally, the system uses UART communication to transmit parameter values and event logs to an external computer for storage, monitoring, and diagnostic analysis.

The system was tested under different operating conditions to verify its functionality. The monitoring mechanism successfully detected abnormal parameter values and triggered the alert system instantly. The event logging mechanism also worked accurately, recording the parameter details along with the corresponding date and time information. These results demonstrate that the system is capable of providing reliable monitoring and fault detection for vehicle applications.

Overall, the Smart Black Box Monitoring System provides a practical and cost-effective solution for improving vehicle safety and diagnostics. By combining real-time parameter monitoring, automatic alert mechanisms, event logging, and communication capabilities, the

system helps detect potential faults at an early stage and provides valuable data for maintenance and analysis. Such monitoring systems are highly beneficial in modern electric vehicles, where electronic control and safety monitoring play a crucial role in maintaining efficient and reliable vehicle performance.

7.2 Future Enhancements

1. **Wireless Communication (Wi-Fi/Bluetooth)** – Enables remote monitoring of vehicle parameters through a mobile application or wireless device, allowing users to receive alerts and system updates in real time.
2. **Cloud-Based Data Storage and Analysis** – Event logs and parameter data can be stored in cloud platforms for long-term monitoring, performance analysis, and predictive maintenance of vehicle systems.
3. **GPS Integration for Vehicle Tracking** – A GPS module can be integrated to record the location of the vehicle when an abnormal event occurs, helping in accident analysis and vehicle tracking.
4. **Mobile Application Interface** – A dedicated mobile application can be developed to display vehicle parameters, alert notifications, and event logs in a user-friendly format.
5. **Advanced Sensor Integration** – Additional sensors such as temperature sensors, vibration sensors, and battery health monitoring modules can be integrated to improve system accuracy and reliability.
6. **AI-Based Fault Prediction** – Artificial Intelligence and machine learning algorithms can be used to analyze recorded data and predict possible vehicle failures before they occur, improving vehicle safety and maintenance efficiency.
7. **Enhanced Data Logging and Storage** – The system can be extended by adding external memory modules such as SD cards to store large amounts of event data for detailed analysis.

These enhancements can further improve the capability, intelligence, and reliability, making it more suitable for advanced electric vehicle monitoring and safety applications.

REFERENCES

- **S. Kumar, R. Sharma, and P. Verma**, "Vehicle Black Box System for Accident Detection and Data Logging," in 2020 International Conference on Intelligent Transportation Systems, 2020, pp. 210–215.
- **Gupta, N. Singh, and M. Patel**, "Smart Vehicle Monitoring System Using Embedded Controllers," in 2019 IEEE International Conference on Embedded Systems and Applications, 2019, pp. 145–150.
- **P. Sharma and V. Rao**, "Event Data Recorder for Vehicle Safety Monitoring," in 2021 International Conference on Smart Transportation and Automotive Systems, 2021, pp. 98–103.
- **Electronics Hub**, How to Use UART Communication in LPC1768 – Tutorial
Link: <https://www.electronicshub.org/lpc1768-uart-tutorial/>
- **Electronics Hub**, How to Use I2C in LPC1768 – LPC1768 I2C Tutorial
Link: <https://www.electronicshub.org/lpc1768-i2c-tutorial/>
- **ARM Limited (2007)**. ARM Cortex-M3 Technical Reference Manual.
- **NXP Semiconductors (2010)**. LPC1768 User Manual.
- **Maxim Integrated (2015)**. DS1307 Real Time Clock Datasheet.
- **Hitachi (2010)**. 16×2 LCD Display Datasheet.
- **Matrix Electronics (2018)**. 4×4 Matrix Keypad Datasheet.
- **Vishay Semiconductors (2019)**. LED Indicator Datasheet.
- **Murata Manufacturing (2017)**. Buzzer Datasheet.

APPENDIX OVERVIEW OF TOOL

Keil uVision IDE

The Keil uVision IDE is a powerful tool for developing embedded applications. Below are the steps for creating a new project, selecting the LPC1768 as the target microcontroller, writing the code, and generating the HEX file:

1. Creating a New Project:

1. Open the Keil uVision IDE and select "Project" -> "New uVision Project."
2. Choose a directory and name for the project, then click "Save."
3. In the "Select Device for Target" window, search for "LPC1768" and select it from the list. Click "OK."

2. Writing the Code:

1. In the Project window, right-click on the "Source Group 1" folder and select "Add New Item to Group."
2. Choose "C File (.c)" and name the file (e.g., "main.c"). Click "Add."
3. Write the code in the editor, using the IDE's syntax highlighting and auto-completion features to assist with coding.

3. Building the Project:

1. Click on the "Build" button (or press F7) to compile the code. The IDE will check for errors and generate the necessary binary files.
2. If there are any errors, they will be displayed in the "Build Output" window. Fix the errors and rebuild the project.

4. Options for Target:

1. Right-click on the project name in the Project window and select "Options for Target."
2. In the "Target" tab, configure the settings for the microcontroller, such as the clock speed and memory layout.
3. In the "Output" tab, ensure that the "Create HEX File" option is checked. This will generate a HEX file that can be flashed onto the microcontroller.

Flash Magic

Flash Magic is a tool used for programming and debugging microcontrollers, including the LPC1768. Below are the steps to use Flash Magic to dump the HEX file onto the LPC1768 microcontroller:

1. Connect the LPC1768 to the Computer:

1. Connect the LPC1768 microcontroller to the computer using a UART interface (e.g., via a USB-to-UART converter). Ensure that the connections are correct, including the TX, RX, and GND pins.

2. Launch Flash Magic:

1. Open the Flash Magic application on your computer. The main interface will display options for selecting the device, communication settings, and the HEX file to be flashed.

3. Select the Device:

1. In the "Device" dropdown menu, select "LPC1768" as the target microcontroller. This ensures that Flash Magic configures the correct settings for the LPC1768.

4. Configure Communication Settings:

1. Set the communication interface to "UART" and configure the baud rate, COM port, and other settings as required. Ensure that the settings match the configuration of your UART interface.

5. Load the HEX File:

1. Click on the "Browse" button to load the HEX file generated by the Keil uVision IDE. This file contains the compiled code that will be flashed onto the microcontroller.

6. Flash the Code:

1. Click on the "Start" button to begin the flashing process. Flash Magic will erase the microcontroller's memory, program the HEX file, and verify the code. Once the process is complete, a success message will be displayed.

7. Verify the Code:

1. After flashing, you can verify that the code has been successfully programmed by checking the microcontroller's behaviour. If the system operates as expected, the flashing process was successful.